

ROS 2 Assignment – Problems Solved and Their Fixes

Environment: ROS 2 Humble in Docker (host: ROS 2 Jazzy) | Gazebo Classic 11 | RViz | Nav2 / AMCL

Purpose: This document records the environment, build, Gazebo/RViz, map-server, AMCL, behavior-tree, and sensor-configuration issues encountered while building the **testbed_navigation** package for this assignment, together with the diagnostic steps and fixes that resolved them — including the localization issue that initially prevented the robot from passing reliably through narrow doorways.

A. Environment, Build & GUI Setup

1. ROS 2 Jazzy host → ROS 2 Humble Docker environment

Problem: The assignment targets ROS 2 Humble and Gazebo Classic, while the existing system runs ROS 2 Jazzy.

Solution: A separate Docker container was created for the assignment so the existing Jazzy installation did not need to change. The workspace was mounted from the host, so source files were edited normally in VS Code while building/testing inside Humble. The repository remains normal ROS package source code — Docker is only the development environment.

```
Host Ubuntu
|-- ROS 2 Jazzy
`-- Docker
    |-- ROS 2 Humble
    |-- /root/ros2_humble_ws <-> ~/ros2_humble_ws
```

2. CMake parse error in testbed_description

Symptom: colcon build --symlink-install reported a CMake parse error near the ament_package call.

Cause: The CMake function was written as **ament_package** instead of **ament_package()**.

Fix:

```
ament_package()
```

3. Gazebo/RViz GUI problems inside Docker

Problem A – display authorization: RViz/Gazebo failed with "Authorization required" and "qt.qpa.xcb: could not connect to display :1". **Cause:** the container had no permission to use the host X11 display. **Fix:** X11 access and Xauthority were passed through from the host.

```
export DISPLAY=:1
# host-side X11 access enabled (xhost)
# container received DISPLAY and XAUTHORITY
```

Problem B – Gazebo crash: After display access was fixed, gzserver crashed with Mesa/DRM messages (failed to access /dev/dri/card1), exit code -11. **Cause:** Gazebo tried to use hardware OpenGL/DRM, which wasn't available through the container's graphics setup. **Fix:** force software OpenGL rendering.

```
export LIBGL_ALWAYS_SOFTWARE=1
```

Non-blocking ALSA audio-device warnings were also observed but were not the cause of the failure.

4. Missing runtime dependencies in the Humble container

Missing item	Observed effect	Fix
--------------	-----------------	-----

xacro	Launch failed with a TextSubstitution / executable-not-found error	apt install ros-humble-xacro
ament_package	CMake could not import ament_package during build	apt install python3-ament-package
nav2_map_server / nav2_ament	ROS could not find the packages	apt install corresponding ROS Humble Nav2 packages
Gazebo Classic binaries	gzclient command missing in the recreated container	apt install Ubuntu Jammy Gazebo (11) + ROS-Gazebo integration packages

B. Map Server & Basic AMCL Setup

5. /map_server could not configure

Symptom: ros2 lifecycle get /map_server returned unconfigured [1]; the configure transition failed.

Cause: testbed_bringup's CMakeLists.txt installed the launch directory but not the maps directory, so testbed_world.yaml / .pgm were missing from the installed share path.

Fix (testbed_bringup/CMakeLists.txt):

```
install(
  DIRECTORY
    launch
    maps
  DESTINATION
    share/${PROJECT_NAME}
)
```

The map YAML was also corrected to reference the actual PGM filename (**testbed_world.pgm**, not the placeholder wrong_path_testbed_world.pgm). After rebuilding, map_server configured and activated correctly.

6. Map topic existed, but RViz showed “No map received”

Diagnostics: /map was confirmed as nav_msgs/msg/OccupancyGrid with one publisher (map_server); RViz was subscribed with matching RELIABLE + TRANSIENT_LOCAL QoS. ros2 topic echo /map --once showed valid metadata (frame_id: map, resolution 0.05, width 405, height 400) — so the map data itself was valid.

Key discovery: AMCL had not yet established the map → odom transform. RViz can subscribe to /map, but the map cannot be positioned in the robot's TF tree until localization has been initialized.

7. AMCL repeatedly requested the initial pose

Symptom: AMCL logged “cannot publish a pose or update the transform. Please set the initial pose...”

TF verification: odometry was healthy — /odom was publishing and odom → base_footprint was available. Only map → odom was missing.

Initial-pose test: publishing a valid /initialpose message made AMCL accept the pose and map → odom appeared, confirming AMCL initialization was the missing step.

Automatic initialization: set_initial_pose: true was added to the AMCL YAML. A filename mismatch (acml_params.yaml vs. amcl_params.yaml referenced in localisation.launch.py) initially prevented the file from being installed/loaded; renaming it to amcl_params.yaml and rebuilding fixed parameter loading.

Correction (updated after further testing): the parameter names originally used below (initial_pose_x / initial_pose_y / initial_pose_a) are ROS 1-style flat keys and are **not** valid Nav2 Humble AMCL parameter names. The correct Humble syntax uses dot-notation sub-keys, which is what the final working configuration actually used:

```
# INCORRECT (does not match Humble AMCL's declared parameters):
amcl:
  ros__parameters:
    set_initial_pose: true
    initial_pose_x: 0.0
    initial_pose_y: 0.0
    initial_pose_a: 0.0

# CORRECT (Humble AMCL dot-notation, confirmed working):
amcl:
  ros__parameters:
    set_initial_pose: true
    initial_pose.x: -0.0138
    initial_pose.y: 4.9945
    initial_pose.z: 0.0
    initial_pose.yaw: -0.0424
```

ros2 param get /amcl set_initial_pose returned True once the correctly named parameters were loaded.

C. Navigation Stack Bring-Up Bugs (found while building testbed_navigation)

The assignment requires manually starting individual Nav2 servers via their own plugins (rather than nav2_bringup), across three launch files: map_loader.launch.py, localisation.launch.py, and navigation.launch.py. The following bugs surfaced while bringing the planner/controller/behavior/BT layer up this way.

8. behavior_server crashed on configure (exit code -11 / SIGSEGV)

Symptom: behavior_server logged “Configuring” and then the process died with exit code -11, no further error text.

Cause: nav2_params.yaml used parameter names local_costmap_topic / global_costmap_topic / local_footprint_topic / global_footprint_topic. On ROS 2 Humble, nav2_behaviors' behavior_server only declares **costmap_topic** and **footprint_topic** (no local/global split) — that four-key schema belongs to a newer Nav2 branch, not Humble. The unrecognized keys were silently ignored, but this mismatch, combined with issue 9 below, produced the crash during plugin configuration.

Fix:

```
behavior_server:
  ros__parameters:
    use_sim_time: true
    costmap_topic: local_costmap/costmap_raw
    footprint_topic: local_costmap/published_footprint
    cycle_frequency: 10.0
    behavior_plugins: [spin, backup, wait]
    global_frame: map
    robot_base_frame: base_footprint
    transform_tolerance: 0.2
```

9. behavior_server fatal plugin load failure (“class does not exist”)

Symptom (after fixing issue 8): behavior_server logged a FATAL pluginlib error: “According to the loaded plugin descriptions the class nav2_behaviors::Spin with base class type nav2_core::Behavior does not exist. Declared types are nav2_behaviors/AssistedTeleop nav2_behaviors/BackUp nav2_behaviors/DriveOnHeading

nav2_behaviors/Spin nav2_behaviors/Wait.”

Cause: the plugin type strings used C++ namespace syntax (**nav2_behaviors::Spin**) instead of the pluginlib-registered name, which uses a forward slash (**nav2_behaviors/Spin**) on this Humble build.

Fix:

```
spin:
  plugin: "nav2_behaviors/Spin"
backup:
  plugin: "nav2_behaviors/BackUp"
wait:
  plugin: "nav2_behaviors/Wait"
```

After both fixes, behavior_server configured successfully and all navigation lifecycle nodes progressed past “configuring”.

10. bt_navigator failed to parse the default behavior tree XML

Symptom: On activation, bt_navigator logged: “Exception when loading BT: Error at line 7: -> Node not recognized: RecoveryNode” while loading navigate_through_poses_w_replanning_and_recovery.xml, and the lifecycle manager aborted bring-up.

Cause: bt_navigator's plugin_lib_names list only registered a subset of BT node plugins (compute-path, follow-path, spin/wait/back-up actions, and several condition nodes). It was missing several plugins the default XML depends on, including the **RecoveryNode** control node, **PipelineSequence**, and rate/distance-controller decorators.

Fix: extended plugin_lib_names to include the full default Humble set, notably:

```
plugin_lib_names:
- nav2_recovery_node_bt_node
- nav2_pipeline_sequence_bt_node
- nav2_rate_controller_bt_node
- nav2_distance_controller_bt_node
- nav2_speed_controller_bt_node
- nav2_round_robin_node_bt_node
- nav2_remove_passed_goals_action_bt_node
- nav2_navigate_to_pose_action_bt_node
- nav2_navigate_through_poses_action_bt_node
# ...plus the entries already present (compute_path_to_pose, follow_path,
#   spin/wait/back_up, clear_costmap_service, goal/path condition nodes, etc.)
```

With the complete plugin list, bt_navigator loaded the default BT XML and activated successfully.

D. Localization Accuracy & the Narrow-Doorway Navigation Failure

This was the most involved issue: the robot would sometimes navigate successfully, but on other runs it drifted off course and ended up appearing to be inside a wall in RViz, or failed to line up with narrow doorways — even though Gazebo's ground-truth view showed it in a perfectly reasonable position. Diagnosis required separating three layered problems.

11. Sparse / near-empty laser scan (root sensor issue)

Symptom: Only a handful of LaserScan points ever rendered in RViz, instead of a dense 500-sample scan, even close to walls.

Cause: the Gazebo ray sensor plugin for lidar_link_1 had **<min_intensity>100.0</min_intensity>**. Gazebo Classic's ray sensor plugin generally does not compute meaningful per-point intensity values (often 0 for all returns),

so this threshold silently discarded nearly every scan point before publishing.

Fix:

```
<min_intensity>0.0</min_intensity>
```

Note: the sensor's `<range><max>1.5</max></range>` (1.5 m) was confirmed to be intentional, not a bug — it matches `obstacle_max_range / raytrace_max_range: 1.5` already set in the `local_costmap` parameters.

12. AMCL confidently localizing to the wrong position (corridor degeneracy)

Symptom: Even after the scan density fix and a correct initial 2D Pose Estimate at launch, the robot's estimated position would still drift away from its true position during a run — and unlike ordinary drift, the AMCL particle cloud looked *tight/converged* rather than spread out, meaning AMCL was confident about a position that was actually wrong.

Diagnosis: `ros2 topic echo /amcl_pose --once` was captured mid-run, at the moment the robot was physically in a doorway in Gazebo but appeared to be at an unrelated corridor intersection in RViz. The pose covariance diagonal was read directly:

```
x variance: 12.876874725539539 (std dev ≈ 3.6 m)
y variance: 0.6568860939092682 (std dev ≈ 0.8 m)
yaw variance: 3.8801911222570813 (std dev ≈ 113 degrees)
```

Tight in y, wildly uncertain in x and yaw — this is the textbook signature of **AMCL degeneracy along a corridor's long axis**: with a 1.5 m-range 2D lidar in a map with multiple visually-identical straight corridor crossings, a single laser scan sees the same two parallel walls no matter where along the corridor (or which way) the robot actually is. Without enough odometry-driven correction, the particle filter can converge confidently onto the wrong instance of a repeated corridor pattern.

Fix: lowered the AMCL odometry noise model parameters (`alpha1`–`alpha4`), which were all at the default 0.2, so that accumulated wheel-odometry motion is trusted more heavily relative to ambiguous single-scan laser matches:

```
amcl:
  ros__parameters:
    alpha1: 0.05 # was 0.2
    alpha2: 0.05 # was 0.2
    alpha3: 0.05 # was 0.2
    alpha4: 0.05 # was 0.2
    alpha5: 0.2  # unchanged
```

Result: after this change, the AMCL particle cloud (visualized via a dedicated ParticleCloud RViz display, since Humble's AMCL publishes `/particle_cloud` as `nav2_msgs/msg/ParticleCloud` rather than `geometry_msgs/msg/PoseArray`) stayed tightly converged and correctly positioned through the corridor. The robot was subsequently able to pass cleanly through the narrow doorway and reach multiple different goal poses consistently across repeated runs, with RViz's estimated position matching Gazebo's ground-truth position throughout.

13. RViz couldn't display the particle cloud (tooling note, not a robot bug)

Symptom: `/particle_cloud` did not appear under RViz's "Add > By topic" tab.

Cause: as noted above, Humble's AMCL publishes `nav2_msgs/msg/ParticleCloud`, a custom message type. RViz only lists a topic under "By topic" if a compatible display plugin is available; the default `rviz_default_plugins` do not include one for this type.

Fix: installed the dedicated display plugin and added it via `display type` instead of `by topic`:

```
sudo apt install ros-humble-nav2-rviz-plugins
```

then: Add -> By display type -> "ParticleCloud" -> set Topic to /particle_cloud

E. Final Working Navigation/Localization Flow

Launch structure

```
Simulation:    testbed_full_bringup.launch.py

Map:           testbed_navigation/map_loader.launch.py
               -> map_server -> lifecycle configure/activate -> /map

Localization:  testbed_navigation/localisation.launch.py
               -> AMCL (corrected alpha1-4, dot-notation initial_pose) -> map -> odom

Navigation:    testbed_navigation/navigation.launch.py
               -> planner_server -> controller_server -> bt_navigator
               -> behavior_server (corrected topic params + plugin names)
               -> lifecycle_manager_navigation
```

TF chain

```
map
  | AMCL (corrected alphas)
  v
odom
  | Gazebo diff-drive odometry
  v
base_footprint
  | static transform
  v
lidar_link_1
```

Reliable startup order used during testing

1. Gazebo + robot 2. Map Server (configure/activate) 3. AMCL / localisation 4. RViz 5. Set/confirm initial AMCL pose 6. Navigation servers (planner/controller/BT/behavior)

14. Final verification checklist

Check	Expected result
ROS distribution	ROS 2 Humble inside Docker
Build	colcon build --symlink-install succeeds
Gazebo GUI	Runs with X11 + software OpenGL
/map_server	active [3]
/map	Publisher: map_server; RViz subscribed
/scan	Dense LaserScan (min_intensity: 0.0); points align with map walls
/odom	nav_msgs/msg/Odometry; odom -> base_footprint TF available
AMCL	Active, initial pose accepted; map -> odom TF available
AMCL particle cloud	Tight/converged cluster throughout a run (checked via nav2_rviz_plugins ParticleCloud display)

/amcl_pose covariance	Low x/y/yaw variance sustained through corridors and doorways, not just at the start pose
behavior_server	Configures/activates without exit -11; spin/backup/wait plugins load
bt_navigator	Loads default BT XML without "Node not recognized" errors
RViz	Displays OccupancyGrid from /map; robot's RViz position matches Gazebo ground truth throughout a run, including through narrow doorways

See BUGS.txt (repository root) for the itemized split between starter-repo bugs and implementation issues, in the format required by the assignment submission.